



SORBONNE UNIVERSITÉ
Master AI2D

**Vers des agents conversationnels
multimodaux contextualisés**

UE de projet M1

Lyna HOCINE – Mariana DUARTE FERREIRA

Encadré par Mahdi KHORAMSHAHI et Hamed RAHIMI

Table des matières

1	Introduction	1
2	État de l’art	1
2.1	Modèles de langage de grande taille (LLMs)	1
2.2	Personnalisation	2
2.3	HARMONI	2
2.4	Model Context Protocol (MCP)	3
3	Contribution	4
3.1	Analyse des besoins	4
3.2	Architecture	5
3.3	Conception du serveur MCP	6
3.3.1	Serveur MCP	6
3.3.2	Agent MCP	7
3.4	Implémentation des outils MCP	9
3.4.1	Outil de recherche Internet	9
3.4.2	Outil HARMONI	9
3.5	Communication	12
3.5.1	Avec HARMONI	12
3.5.2	Gestion des inputs multimodaux	12
3.6	Tests et validation	13
3.6.1	Exemples de scénarios de test	14
4	Résultats et discussion	17
4.1	Résultats obtenus	17
4.2	Avantages	18
4.3	Limites	18
4.4	Perspectives	18
5	Conclusion	19
A	Cahier des charges	
B	Manuel utilisateur	

1 Introduction

L'intelligence artificielle connaît aujourd'hui une évolution rapide grâce au développement des modèles de langage de grande taille (LLMs) et des systèmes multimodaux capables de traiter différents types de données comme le texte, l'audio ou encore les images. Ces modèles sont désormais utilisés dans de nombreux domaines tels que les assistants virtuels, la robotique, les systèmes conversationnels ou les interfaces intelligentes. Cependant, malgré leurs capacités avancées, les modèles de langage restent limités lorsqu'ils doivent interagir avec des environnements réels et dynamiques. En effet, ils ne disposent pas nativement d'un accès direct à des systèmes externes de perception, de mémoire ou de traitement multimodal.

Dans le domaine des interactions humain-robot, ces limitations deviennent particulièrement importantes. Les robots sociaux doivent être capables d'identifier plusieurs utilisateurs, de mémoriser des informations à long terme, d'analyser des flux audio et vidéo, et de générer des réponses personnalisées en fonction du contexte. Pour répondre à ces besoins, le framework HARMONI a été proposé comme une solution multimodale permettant de gérer des interactions personnalisées entre humains et robots grâce à l'utilisation de modèles de langage et de différents modules de perception et de mémoire. Le framework intègre notamment des mécanismes de reconnaissance utilisateur, de modélisation du contexte et de gestion de profils utilisateurs multimodaux.

Parallèlement, le Model Context Protocol (MCP) est apparu comme une nouvelle approche permettant aux modèles de langage d'interagir avec des outils et des services externes de manière standardisée. MCP propose une architecture générique basée sur des outils exposés via des interfaces structurées, facilitant ainsi l'interopérabilité entre différents modèles d'intelligence artificielle et systèmes logiciels.

Dans ce contexte, transformer HARMONI en un outil compatible MCP présente plusieurs avantages. Premièrement, cela permet de rendre ses fonctionnalités multimodales accessibles à n'importe quel agent compatible MCP, sans nécessiter une intégration spécifique du framework. Deuxièmement, cette approche améliore l'interopérabilité et la modularité du système en séparant les capacités multimodales de la logique de l'agent conversationnel. Enfin, elle facilite la réutilisation et l'extensibilité de HARMONI dans différents environnements applicatifs, notamment les assistants intelligents, les systèmes robotiques et les plateformes conversationnelles.

L'objectif de notre projet n'était donc pas de reconstruire le framework HARMONI, mais de développer une couche d'intégration permettant d'exposer ses fonctionnalités sous forme d'outil MCP. Cette transformation permet à un LLM d'exploiter les capacités de mémoire, de perception et de récupération de contexte HARMONI via une interface standardisée et réutilisable.

2 État de l'art

2.1 Modèles de langage de grande taille (LLMs)

Un grand modèle de langage (Large Language Model - LLM) est un réseau de neurones entraîné sur une très grande quantité de données textuelles afin de réaliser plusieurs tâches de traitement automatique du langage naturel. Les LLMs sont capables de générer, résumer,

traduire et analyser du texte dans différents contextes, et constituent aujourd’hui une technologie fondamentale utilisée dans les assistants conversationnels modernes.

Cependant, ces modèles présentent des limites importantes : ils ne disposent pas de mémoire persistante et n’ont pas de perception en temps réel de leur environnement. De plus, ils rencontrent des difficultés lorsqu’ils doivent interagir avec des environnements dynamiques et ne possèdent pas d’accès natif à des outils ou services externes.

Pour dépasser ces limites, plusieurs approches ont été proposées, notamment l’utilisation des outils externes permettant aux modèles d’accéder à des systèmes de mémoire, de perception ou à des services spécialisés.

2.2 Personnalisation

La personnalisation des systèmes de dialogue constitue aujourd’hui un axe de recherche important visant à rendre les interactions humain-machine plus naturelles et pertinentes. Plusieurs travaux abordent cette problématique sous différents angles : l’analyse des jeux de données et des catégories de personnalisation [1], les framework comme *In-Dialogue Learning (IDL)* [2] capables d’inférer une personnalité à partir de l’historique des conversations, ou encore des modèles comme *MIRACLE* [3] qui représentent les traits utilisateurs sous forme d’attributs multiples.

D’autres approches, telles que *Persona-Adaptive Attention (PAA)* [4], combinent les informations de personnalité et le contexte conversationnel afin de produire des réponses plus pertinentes tout en réduisant les redondances. Enfin, les techniques de *prompt-tuning* [5] permettent une personnalisation efficace et multilingue des grands modèles pré-entraînés.

Malgré ces avancées, plusieurs défis persistent dans les environnements multi-utilisateurs et les systèmes d’assistance sociale, notamment concernant la confidentialité, la sécurité éthique et la gestion de la mémoire à long terme.

2.3 HARMONI

Les interactions humain-robot en milieu partagé impliquent souvent plusieurs utilisateurs simultanément, ce qui introduit des difficultés spécifiques : identifier le locuteur actif, mémoriser les préférences de chacun, gérer les interruptions, et adapter le comportement du robot sans perdre le fil de la conversation. Or, la plupart des systèmes existants ne proposent qu’une personnalisation limitée à un seul utilisateur ou n’intègrent pas de mémoire à long terme permettant de reconnaître les interlocuteurs d’une session à l’autre. De plus, la coordination entre la perception multimodale (audio, vidéo) et la génération de réponses personnalisées reste souvent traitée de manière fragmentée.

C’est dans ce contexte que s’inscrit HARMONI [6]. Contrairement aux approches classiques, HARMONI combine en un seul cadre modulaire quatre composants clés :

- Module de perception qui traite les flux audiovisuels pour identifier le locuteur actif et extraire les entrées multimodales
- Module de modélisation du monde qui maintient le contexte conversationnel à court terme ainsi que l’état de l’environnement

- Module de modélisation de l'utilisateur qui stocke et met à jour des profils à long terme spécifiques à chaque locuteur
- Module de génération qui produit des réponses éthiques, personnalisées et adaptées au contexte

Cette architecture permet au robot de s'adapter en temps réel à plusieurs utilisateurs, de conserver un historique de chaque interaction, et de produire des réponses à la fois contextualisées et respectueuses de la vie privée. Ainsi, HARMONI comble un manque identifié dans les systèmes actuels : offrir une personnalisation continue et multi-utilisateur dans des environnements sociaux dynamiques.

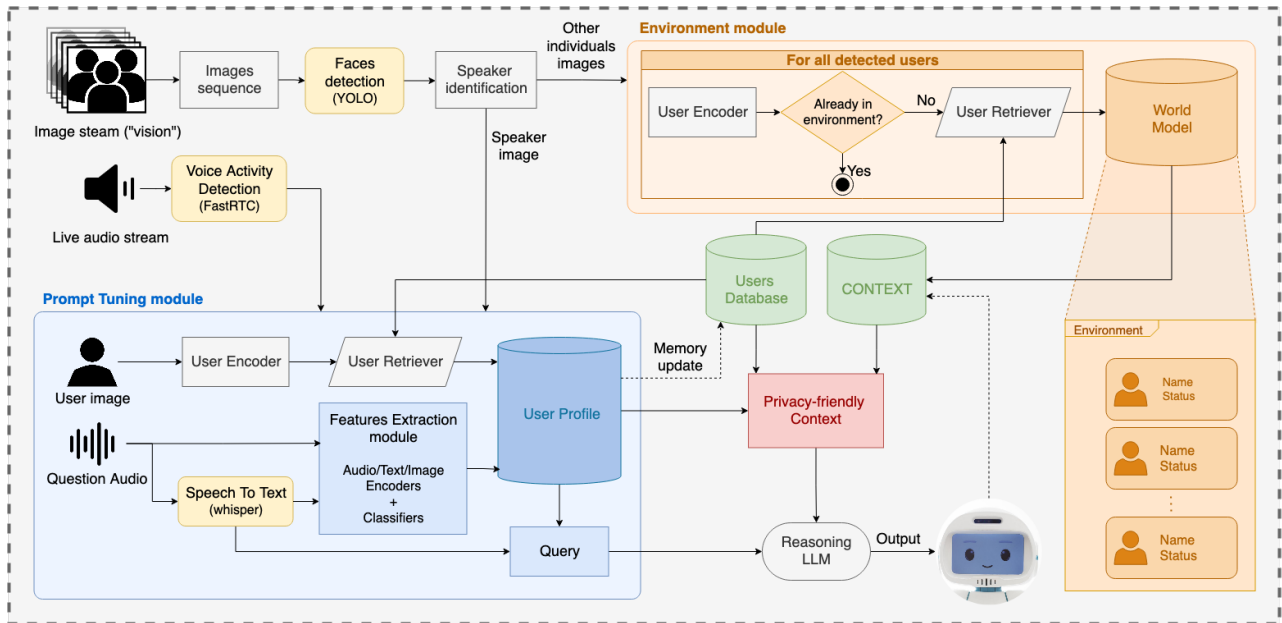


FIGURE 1 – Architecture du framework HARMONI

Malgré les capacités avancées proposées par HARMONI, son utilisation reste principalement liée à son propre environnement logiciel. Les fonctionnalités de perception, de mémoire et de personnalisation qu'il fournit ne sont pas directement accessibles aux agents conversationnels modernes ou aux autres systèmes basés sur des LLMs. Cela limite la réutilisation du framework ainsi que son intégration dans des architectures d'agents plus génériques et modulaires.

Afin de rendre ces capacités multimodales plus facilement exploitables et interoperables avec différents systèmes d'intelligence artificielle, il devient nécessaire de disposer d'un mécanisme standardisé permettant à un modèle de langage d'interagir avec des outils externes. C'est dans cette logique que s'introduit le *Model Context Protocol* (MCP), présenté dans la section suivante.

2.4 Model Context Protocol (MCP)

Le *Model Context Protocol* (MCP) [7] est un protocole standard conçu pour permettre à un modèle de langage d'interagir avec des outils externes via des interfaces structurées. Il vise à faciliter la connexion entre les modèles et différentes sources de données ou services, comme des fichiers locaux, des bases de données ou des API.

L'intérêt principal du MCP est de standardiser ces interactions, afin d'éviter des intégrations spécifiques pour chaque outil. Grâce à ce cadre commun, les applications d'intelligence artificielle deviennent plus modulaires et plus faciles à étendre, car de nouveaux outils peuvent être ajoutés sans modifier le fonctionnement du modèle lui-même.

Concrètement, le MCP repose sur un échange structuré entre le modèle et des connecteurs d'outils, qui exécutent les actions demandées et renvoient les résultats dans un format exploitable. Cela permet au modèle d'enrichir ses réponses avec des données externes et d'effectuer certaines tâches de manière dynamique.

Dans le cadre des systèmes d'IA modernes, le MCP s'inscrit dans une logique d'agents augmentés, capables non seulement de générer du texte mais aussi d'interagir avec leur environnement. Il constitue ainsi une base intéressante pour l'intégration de frameworks permettant d'exploiter des outils externes de manière standardisée.

Par rapport aux approches classiques d'intégration, le MCP apporte plusieurs avantages en terme de modularité, d'interopérabilité et de réutilisabilité des outils.

TABLE 1 – Comparaison entre intégration traditionnelle et MCP

Critère	Intégration traditionnelle	Approche basée sur MCP
Couplage avec le système	Forte dépendance à l'architecture interne du framework	Architecture plus modulaire et découplée
Réutilisabilité	Intégration spécifique à chaque application	Outils réutilisables par différents agents compatibles MCP
Interopérabilité	Faible compatibilité entre systèmes différents	Standard commun facilitant l'interopérabilité
Ajout de nouvelles fonctionnalités	Nécessite souvent des modifications importantes	Ajout simplifié de nouveaux outils et services
Maintenance	Plus complexe avec plusieurs intégrations spécifiques	Maintenance facilitée grâce à une interface standardisée
Intégration avec les LLMs	Dépend d'implémentations personnalisées	Connexion directe via le protocole MCP

Cette approche permet donc d'intégrer plus facilement les fonctionnalités avancées de HARMONI dans des systèmes d'agents modernes, tout en évitant les dépendances fortes à une architecture spécifique et les intégrations propriétaires.

3 Contribution

3.1 Analyse des besoins

Les modèles de langage modernes possèdent aujourd'hui des capacités avancées de génération et de compréhension du langage naturel. Cependant, ils restent limités lorsqu'ils doivent interagir avec des environnements réels et dynamiques. En effet, les LLMs ne disposent pas nativement de mémoire persistante, de mécanismes de perception multimodale ou d'un accès direct à des

outils et systèmes externes.

Le framework HARMONI répond en partie à ces problématiques grâce à son architecture multimodale permettant la gestion de profils utilisateurs, la récupération de contexte conversationnel, ainsi que l'exploitation de données audio et visuelles. Toutefois, HARMONI a été conçu comme un framework indépendant et ses fonctionnalités ne sont pas directement accessibles par des modèles de langage externes de manière standardisée.

Par ailleurs, avec l'émergence des architectures basées sur le *tool calling* et les agents d'intelligence artificielle, le besoin d'interopérabilité entre les modèles et les systèmes externes devient de plus en plus important. Dans ce contexte, le protocole MCP apporte une solution standardisée permettant d'exposer des fonctionnalités sous forme d'outils réutilisables par différents modèles d'intelligence artificielle.

L'objectif principal de notre projet est donc d'intégrer HARMONI au protocole MCP afin de transformer ses fonctionnalités multimodales en outils compatibles MCP. Cette intégration permet à des modèles de langage ou des modèles multimodaux d'accéder aux capacités de perception, de mémoire utilisateur et de récupération de contexte fourni par HARMONI à travers une interface standardisée et extensible.

3.2 Architecture

L'architecture mise en place dans le cadre de notre projet repose sur l'utilisation d'un agent MCP jouant le rôle d'intermédiaire entre l'utilisateur et les différents outils externes. Cette approche permet de centraliser les échanges tout en exposant les fonctionnalités du framework HARMONI sous forme d'outils compatibles avec le protocole MCP. Le fonctionnement général de l'architecture est présenté dans la figure 2.

Lorsqu'un utilisateur envoie une requête, celle-ci est d'abord reçue par l'agent MCP. Ce dernier analyse alors la demande afin d'identifier l'outil le plus adapté pour y répondre.

Si la requête nécessite des capacités multimodales ou l'exploitation de données liées à la mémoire utilisateur, l'agent MCP interagit avec l'outil HARMONI. Celui-ci mobilise alors les différents modules du framework, tels que les mécanismes de perception, la récupération de contexte ou encore la gestion des profils utilisateurs.

L'architecture offre également la possibilité d'intégrer divers services externes, tels qu'un moteur de recherche Internet. Selon la nature de la requête, l'agent MCP peut faire appel à plusieurs outils, combiner les informations obtenues et produire une réponse adaptée à l'utilisateur.

Cette organisation présente plusieurs avantages. Elle favorise une séparation claire entre la logique du modèle de langage et les outils exploités, tout en garantissant une architecture modulaire et extensible. De nouveaux services peuvent être ajoutés sans modifier le fonctionnement global de l'agent MCP. Enfin, l'utilisation du protocole MCP améliore l'interopérabilité entre différents modèles et services externes.

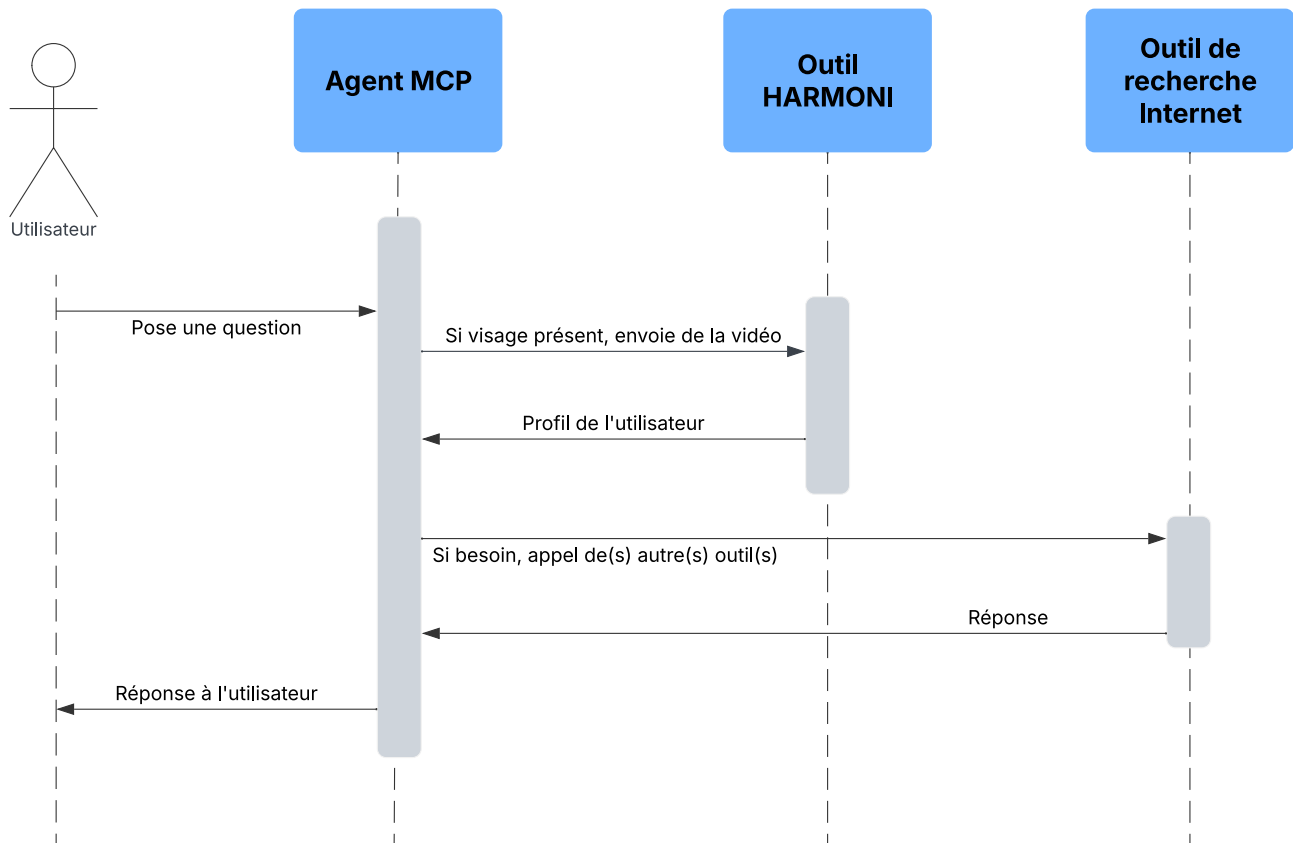


FIGURE 2 – Diagramme de séquence de l'interaction Utilisateur - Agent MCP

3.3 Conception du serveur MCP

La conception de l'intégration repose sur deux composants principaux : le serveur MCP et l'agent MCP. Le serveur MCP a pour rôle d'exposer les fonctionnalités disponibles sous forme d'outils compatibles avec le protocole MCP, tandis que l'agent MCP se charge d'interpréter la requête utilisateur, de sélectionner les outils appropriés et de produire une réponse finale.

3.3.1 Serveur MCP

Le serveur MCP est implémenté à l'aide de la classe `FastMCP`. Le fragment de code suivant montre l'initialisation du serveur MCP :

```

1 from mcp.server.fastmcp import FastMCP
2
3 mcp = FastMCP("mcp-server")

```

Listing 1 – Initialisation du serveur MCP

Le serveur permet de déclarer plusieurs outils accessibles par l'agent à travers le protocole MCP. Chaque outil est défini à l'aide du décorateur `@mcp.tool()`, ce qui permet de transformer une fonction Python classique en outil invocable par un modèle de langage.

Dans notre implémentation, le serveur expose plusieurs outils correspondant à différents besoins :

- `call_web_search_tool` : permet d'effectuer une recherche web à partir d'une requête textuelle.

- `call_image_processing_tool` : permet d'analyser une image à l'aide d'un modèle vision-langage.
- `call_harmoni_tool` : permet d'envoyer une vidéo au framework HARMONI afin de récupérer un contexte utilisateur personnalisé.
- `detect_face` : permet de détecter la présence d'un visage dans une vidéo à l'aide d'OpenCV.
- `transcribe_video` : permet de transcrire l'audio d'une vidéo à l'aide du modèle Whisper.

L'extrait suivant montre comment une fonction Python classique peut être exposée comme outil MCP grâce au décorateur `@mcp.tool()` :

```

1 @mcp.tool()
2 def call_web_search_tool(query: str) -> str:
3     """
4     calls the web search tool and returns a response
5     """
6     response_dic = WebSearchTool.web_search_tool(query)
7     return response_dic["result"]

```

Listing 2 – Déclaration d'un outil MCP

Le serveur inclut également un outil de détection de visage basé sur OpenCV. Celui-ci permet de déterminer si une vidéo contient un locuteur visible avant d'invoquer le framework HARMONI.

```

1 cap = cv2.VideoCapture(video_path)
2
3 face_cascade = cv2.CascadeClassifier(
4     cv2.data.harcascades +
5     'haarcascade_frontalface_default.xml'
6 )
7
8 faces = face_cascade.detectMultiScale(
9     gray,
10    1.1,
11    4
12 )

```

Listing 3 – Détection de visage avec OpenCV

Ce mécanisme permet d'identifier rapidement si une interaction vidéo contient un visage humain avant de décider si le framework HARMONI doit être utilisé pour la personnalisation multi-utilisateurs.

Le serveur joue donc le rôle de couche d'abstraction entre les fonctionnalités techniques et le modèle de langage. L'agent n'a pas besoin de connaître l'implémentation interne de chaque outil : il reçoit simplement leur description, leurs paramètres d'entrée et leur résultat de sortie. Cette séparation permet de rendre le système plus modulaire et plus facilement extensible.

3.3.2 Agent MCP

L'agent MCP représente la partie intelligente du système. Il utilise un modèle de langage local via ChatOllama [8] et se connecte au serveur MCP à l'aide de `MultiServerMCPClient`. Le fragment suivant montre la connexion au serveur MCP et la création de l'agent ReAct.

```

1 client = MultiServerMCPClient({
2     "web-search": {
3         "command": "python",
4         "args": [SERVER_FILE],
5         "transport": "stdio",
6         "cwd": SERVER_DIR,
7     }
8 })
9
10 tools = await client.get_tools()
11
12 self.agent = create_react_agent(
13     self.model,
14     tools
15 )

```

Listing 4 – Connexion au serveur MCP

Après l'initialisation, l'agent récupère automatiquement la liste des outils exposés par le serveur, puis les utilise dans un agent de type ReAct [9] créé avec `create_react_agent` issu de LangChain [10].

Le rôle principal de l'agent est d'analyser la requête de l'utilisateur et de décider quel outil doit être appelé. Pour guider son comportement, un `system_prompt` est défini. Ce prompt précise notamment la manière dont les fichiers vidéo doivent être traités.

```

1 self.system_prompt = SystemMessage(content=(
2     "When the user's message contains a video: "
3     "1. Call detect_face. "
4     "2. If True, call call_harmoni_tool. "
5     "3. Otherwise call transcribe_video."
6 ))

```

Listing 5 – Prompt système de l'agent MCP

Le fonctionnement de l'agent suit donc une logique conditionnelle : lorsqu'une vidéo est fournie, l'outil `detect_face` est appelé en premier. Si un visage est détecté, l'agent invoque ensuite `call_harmoni_tool` afin d'obtenir un contexte utilisateur personnalisé. Dans le cas contraire, il utilise l'outil `transcribe_video` pour extraire uniquement le contenu audio de la vidéo.

L'utilisateur peut interagir avec le système soit par texte, soit par voix grâce au module de transcription vocale intégré au système.

Ainsi, l'agent MCP agit comme un orchestrateur : il reçoit la demande utilisateur, choisit les outils nécessaires, récupère leurs résultats, puis formule une réponse finale adaptée au contexte.

3.4 Implémentation des outils MCP

3.4.1 Outil de recherche Internet

Afin de nous familiariser avec le MCP, notre premier développement a été un outil de recherche Internet. Cela nous a permis de comprendre en détail l'architecture d'un agent MCP ainsi que le processus d'appel d'un outil. Cette implémentation repose sur une classe `WebSearchTool` qui expose deux méthodes principales.

La méthode `process_input` détecte le type d'entrée utilisateur : si celle-ci correspond à un fichier audio (`.wav`), elle appelle un module de reconnaissance vocale (`SpeechToText`) pour le transcrire en texte ; si l'entrée est déjà du texte, elle est renvoyée telle quelle (les images étant traitées directement par le LLM).

La méthode `web_search_tool` prend une requête (pouvant être multimodale), la convertit en texte via `process_input`, puis interroge l'API `SerpApi` (avec une clé d'API stockée dans les variables d'environnement) pour récupérer le premier résultat organique de Google. Le résultat est formaté sous forme de titre et d'extrait, puis retourné à l'agent. Ce mécanisme illustre concrètement comment un outil MCP peut encapsuler une fonction externe (recherche Web) et être invoqué par un agent LLM pour enrichir ses capacités.

3.4.2 Outil HARMONI

L'objectif principal de notre projet était de développer un outil permettant d'exploiter HARMONI au sein d'un agent basé sur le Model Context Protocol. Pour cela, nous avons créé la classe `HarmoniTool` avec des appels de base à l'API REST de HARMONI.

```
1 import threading
2 import requests
3
4 HARMONI_BASE_URL = "http://127.0.0.1:8000"
5
6 class HarmoniTool:
7
8     @staticmethod
9     def harmoni_tool(video_path: str, query: str) -> str:
```

Listing 6 – Structure générale de l'outil HARMONI

Adopter une API REST nous a permis d'intégrer facilement et de manière modulaire le framework HARMONI dans notre agent MCP, sans avoir à réécrire ses modules complexes de perception et de mémorisation, tout en bénéficiant d'une communication standardisée et asynchrone.

Architecture de l'outil HARMONI. Nous avons fait le choix de concevoir un unique outil regroupant l'ensemble des fonctionnalités du framework HARMONI, plutôt que de créer un outil distinct pour chaque module (perception, mémoire, profil utilisateur, etc.). Ce choix architectural repose sur le fait que, dans notre cas d'usage, chaque interaction utilisateur nécessite systématiquement l'exploitation combinée de tous les modules de HARMONI. Lorsqu'une vidéo est transmise au framework, les différentes étapes de traitement sont fortement dépendantes les unes des autres.

Créer plusieurs outils indépendants aurait augmenté la complexité des échanges et la latence des traitements. Nous avons donc choisi de regrouper les fonctionnalités de HARMONI dans un unique outil, permettant à l'agent MCP d'obtenir directement un contexte utilisateur complet à partir d'une seule requête.

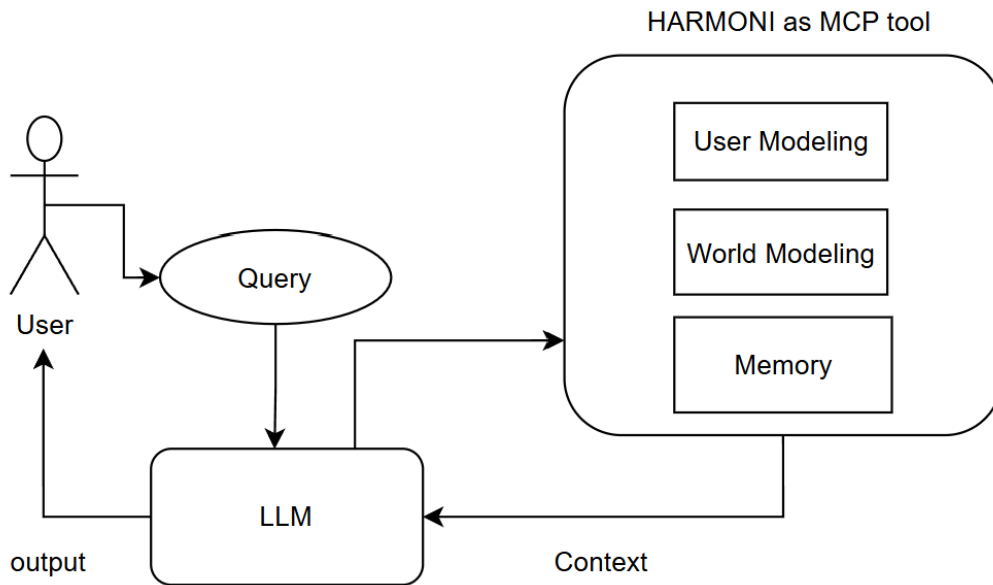


FIGURE 3 – Structure du système de HARMONI en tant qu'outil MCP

La classe `HarmoniTool` contient une méthode statique `harmoni_tool(video_path, query)`. Elle reçoit le chemin d'un fichier vidéo et une requête textuelle. Le fonctionnement se décompose en trois phases.

1. Envoi de la vidéo à HARMONI. La méthode ouvre le fichier vidéo en lecture binaire et envoie une requête HTTP POST à l'endpoint `/set_video` de l'API locale (par défaut `http://127.0.0.1:8000`).

```

1 with open(video_path, "rb") as f:
2     response = requests.post(
3         f"{HARMONI_BASE_URL}/set_video",
4         files={"video": f}
5     )
6
7 if response.status_code != 200:
8     return f"Harmoni /set_video failed: {response.status_code} - {
9         response.text}"
10
11 data = response.json()
12
13 detected_user = data.get("detected_user", "unknown")
14 profile = data.get("profile", [])
15 emotion = data.get("emotion", "unknown")
16 transcript = data.get("transcript", "")

```

Listing 7 – Envoi de la vidéo à l’API HARMONI

Cette étape déclenche le module de perception de HARMONI : détection du locuteur actif, transcription audio, extraction de l’émotion et récupération du profil utilisateur (âge, genre, centre d’intérêt, etc.). La réponse JSON contient les champs `detected_user`, `profile` (une liste de paires nom/valeur), `emotion` et `transcript`.

2. Mise à jour asynchrone de la mémoire. Conformément au fonctionnement de HARMONI, chaque interaction doit mettre à jour le profil long-terme de l’utilisateur avec les nouvelles informations. Pour ne pas bloquer la réponse de notre outil, nous appelons l’endpoint `/answer` de l’API dans un `thread` séparé.

```
1 def _update_memory():
2     requests.post(
3         f"{HARMONI_BASE_URL}/answer",
4         data={
5             "emotion": emotion,
6             "current_user": detected_user,
7             "question": transcript,
8         }
9     )
10
11 threading.Thread(
12     target=_update_memory,
13     daemon=True
14 ).start()
```

Listing 8 – Mise à jour asynchrone de la mémoire utilisateur

Cette requête envoie l’émotion, l’identifiant de l’utilisateur et la transcription. Nous ignorons la réponse car la mise à jour est effectuée en arrière-plan ; ainsi l’agent n’attend pas la fin de cette opération pour poursuivre.

3. Formatage du contexte utilisateur. L’information la plus utile pour notre agent n’est pas la réponse textuelle générée par HARMONI (qui serait redondante avec ce que l’agent produit lui-même), mais bien le contexte utilisateur extrait : identité, émotion, transcription, et surtout le profil. Nous construisons donc une chaîne de caractères structurée :

```
1 lines = [
2     f"Detected user: {detected_user}",
3     f"Emotion: {emotion}",
4     f"Transcript: {transcript}",
5     f"Profile:",
6 ]
7
8 if profile:
9     for feature in profile:
10         lines.append(
11             f"- {feature.get('name')}: {feature.get('value')}"
12         )
13 else:
```

```

14     lines.append("No profile data found.")
15
16 return "\n".join(lines)

```

Listing 9 – Construction du contexte utilisateur

Le contexte contient :

- **Detected user**: nom de l'utilisateur reconnu,
- **Emotion**: état émotionnel détecté,
- **Transcript**: requête transcrite,
- **Profile**: liste des caractéristiques utilisateur.

Si aucun profil n'est trouvé, un message par défaut est inséré. Cette chaîne est retournée par l'outil et sera injectée dans le prompt de l'agent MCP pour personnaliser sa réponse.

En résumé, notre outil HARMONI agit comme un pont entre l'agent MCP et le framework HARMONI. La classe `HarmoniTool` utilise un thread séparé pour ne pas dégrader la latence, extrait les informations utiles des réponses JSON (utilisateur détecté, émotion, transcription, profil), les formate clairement pour l'agent, et gère les erreurs réseau. Grâce à cet outil, l'agent peut bénéficier en temps réel de la détection du locuteur, de la mémoire long-terme et de la personnalisation multi-utilisateurs de HARMONI, sans avoir à implémenter lui-même ces composants complexes.

3.5 Communication

3.5.1 Avec HARMONI

La communication entre l'agent MCP et le framework HARMONI repose sur une API REST locale. Ce choix permet de découpler complètement l'agent du fonctionnement interne de HARMONI, tout en assurant une communication standardisée et facilement extensible.

Les échanges sont réalisés via des requêtes HTTP POST envoyées aux différents endpoints exposés par HARMONI. Lorsqu'une vidéo est transmise à l'outil `HarmoniTool`, celle-ci est envoyée à l'endpoint `/set_video`, qui déclenche les différents modules du framework, notamment la perception multimodale, l'identification utilisateur et la récupération du profil mémorisé.

Les réponses retournées par HARMONI sont au format JSON et contiennent plusieurs informations exploitables par l'agent, comme l'utilisateur détecté, la transcription audio, l'émotion identifiée ainsi que les données du profil utilisateur. Ces informations sont ensuite reformattées afin d'être injectées dans le contexte du modèle de langage.

Enfin, certaines opérations, comme la mise à jour de la mémoire long-terme, sont exécutées de manière asynchrone grâce à l'utilisation de threads. Cette approche permet de réduire la latence perçue par l'utilisateur tout en conservant les mécanismes de mémorisation du framework HARMONI.

3.5.2 Gestion des inputs multimodaux

Notre système prend en charge plusieurs types d'entrées utilisateur : texte, audio, image et vidéo. Chaque type d'entrée nécessite un traitement spécifique avant d'être exploité par l'agent

MCP.

Les requêtes textuelles sont directement transmises au modèle de langage. Les fichiers audio sont d'abord transcrits en texte grâce à un module de reconnaissance vocale basé sur Whisper. Les images peuvent être analysées à l'aide d'un modèle vision-langage afin d'extraire leur contenu sémantique. Enfin, les vidéos sont traitées soit par l'outil HARMONI lorsqu'un visage est détecté, soit par un simple module de transcription audio dans le cas contraire.

Le choix du traitement est effectué dynamiquement par l'agent MCP à partir des informations disponibles dans le contexte et des instructions définies dans le `system_prompt`. Cette approche permet d'adapter automatiquement le pipeline de traitement en fonction du type d'entrée et des besoins de la requête utilisateur.

3.6 Tests et validation

Plusieurs tests ont été réalisés afin de valider le bon fonctionnement de notre projet. Les résultats obtenus montrent que l'agent MCP prend correctement en charge différents types d'entrées, qu'il s'agisse de texte, d'images, de vidéos ou encore d'interactions vocales.

Grâce aux informations disponibles dans le contexte, l'agent est capable d'identifier les outils pertinents à utiliser pour répondre à une requête donnée. Il sélectionne et invoque uniquement les outils nécessaires, ce qui permet d'optimiser le traitement des requêtes et de limiter les appels inutiles.

L'outil HARMONI a également été validé avec succès. Les tests ont confirmé sa capacité à récupérer un profil utilisateur existant lorsqu'il est déjà enregistré dans la mémoire du framework, ou à créer un nouveau profil lorsqu'aucune information préalable n'est disponible. Ce comportement est conforme au fonctionnement attendu du framework HARMONI et démontre l'intégration correcte des mécanismes de mémoire et de personnalisation au sein de notre agent MCP.

Afin d'évaluer les performances du système, une série de tests a été réalisée pour analyser le comportement de l'agent dans différents scénarios. Les tests ont été réalisés en utilisant un modèle de langage local basé sur ChatOllama avec la configuration suivante : `model = "gpt-oss:20b"`, `temperature = 0`. L'agent reçoit uniquement le chemin de la vidéo en entrée, sans instructions spécifiques sur les outils à utiliser. La sélection et l'enchaînement des outils sont entièrement laissés à la responsabilité de l'agent MCP.

Afin de valider le comportement du système dans différents contextes, chaque scénario a été exécuté 10 fois. Trois types de tests ont été réalisés, les scénarios correspondants étant présentés par la suite.

TABLE 2 – Résultats expérimentaux

Description du test	Succès / Tests	Résultats obtenus
Détection correcte d'un visage dans une vidéo	10 / 10	Le système détecte correctement la présence d'un visage dans toutes vidéos testées.
Transcription audio d'une vidéo sans visage	9 / 10	Le contenu audio est correctement transcrit avec peu d'erreurs.
Utilisation des outils MCP par l'agent	7 / 10	La plupart des fois l'agent sélectionne correctement les outils selon le contexte. Or, dans certains cas, il se limite à exécuter uniquement l'outil de détection de visage sans poursuivre avec l'appel à HARMONI lorsqu'un visage est présent.

Les résultats présentés dans le tableau correspondent aux taux de réussite observés sur ces 10 exécutions pour chaque scénario.

3.6.1 Exemples de scénarios de test

Afin de valider le bon fonctionnement de l'agent MCP et de l'intégration de HARMONI, plusieurs scénarios de test ont été réalisés.

Scénario 1 : Récupération du profil utilisateur

Dans ce premier scénario, un utilisateur déjà connu transmet une vidéo contenant une question simple : *What is my name ?*.

L'agent MCP commence par appeler l'outil `detect_face` afin de vérifier la présence d'un visage dans la vidéo. Après détection positive, l'outil `call_harmoni_tool` est invoqué pour récupérer le contexte utilisateur à partir du framework HARMONI.

Le framework retourne alors plusieurs informations :

- l'utilisateur détecté
- la transcription de la vidéo
- l'émotion détectée
- le profil mémorisé associé à cet utilisateur

Grâce à ces informations, l'agent est capable de répondre correctement en utilisant les données mémorisées dans le profil utilisateur.

```

USER: "C:\Users\marid\Pictures\Camera Roll\WIN_20260518_14_33_26_Pro.mp4"

TOOL CALLS:
→ detect_face {'video_path': 'C:\\Users\\marid\\Pictures\\Camera Roll\\WIN_20260518_14_33_26_Pro.mp4'}

TOOL RESPONSE: [{'type': 'text', 'text': 'true', 'id': 'lc_0b21ebaa-6c44-4bd4-96c9-82b3e75edb4c'}]

TOOL CALLS:
→ call_harmoni_tool {'query': 'Hello Mariana, how can I help you?', 'video_path': 'C:\\Users\\marid\\Pictures\\Camera Roll\\WIN_20260518_14_33_26_Pro.mp4'}

TOOL RESPONSE: [{'type': 'text', 'text': 'Detected user: user9\nEmotion: neutral\nTranscript: What is my name?\nProfile:\n- nom: Mariana\n- age: \n- genre: \n- preference_dialogue: ', 'id': 'lc_7cc84dee-93fb-4bbb-9f95-9a020846ecfa'}]

AI: You're Mariana!

[Total: 189.62s]

```

FIGURE 4 – Exemple de récupération du profil utilisateur via HARMONI

Scénario 2 : Combinaison HARMONI + recherche web

Dans ce second scénario, l'utilisateur pose une question concernant la météo à Paris à travers une vidéo.

L'agent détecte d'abord un visage dans la vidéo, puis utilise HARMONI afin d'identifier l'utilisateur et récupérer son contexte personnalisé. Ensuite, l'agent décide dynamiquement d'appeler l'outil de recherche web afin d'obtenir des informations météo actualisées.

Ce scénario montre que l'agent MCP est capable de combiner plusieurs outils dans une même chaîne de traitement : HARMONI pour la personnalisation utilisateur, puis un outil externe de recherche Web pour enrichir la réponse avec des données temps réel.

```

USER: "C:\Users\marid\Pictures\Camera Roll\WIN_20260518_13_58_32_Pro.mp4"

TOOL CALLS:
→ detect_face {'video_path': 'C:\\Users\\marid\\Pictures\\Camera Roll\\WIN_20260518_13_58_32_Pro.mp4'}

TOOL RESPONSE: [{'type': 'text', 'text': 'true', 'id': '1c_b75f15bb-3325-4acc-937b-7521c9a7bd41'}]

TOOL CALLS:
→ call_harmoni_tool {'query': 'C:\\Users\\marid\\Pictures\\Camera Roll\\WIN_20260518_13_58_32_Pro.mp4', 'video_path': 'C:\\Users\\marid\\Pictures\\Camera Roll\\WIN_20260518_13_58_32_Pro.mp4'}

TOOL RESPONSE: [{'type': 'text', 'text': 'Detected user: user9\nEmotion: neutral\nTranscript: How is the weather in Paris today?\nProfile:\n- nom: Mariana\n- age: \n- genre: \n- preference_dialogue: ', 'id': '1c_6c6a3bdd-eabe-4812-8622-af3125200606'}]

TOOL CALLS:
→ call_web_search_tool {'query': 'weather in Paris today'}

TOOL RESPONSE: [{'type': 'text', 'text': "Weather Forecast and Conditions for Paris, France\nToday's and tonight's Paris, France weather forecast, weather conditions and Doppler radar from The Weather Channel and weather.com.", 'id': '1c_76380564-3e8e-46c8-84ee-32c714fef95c'}]

AI: Hi Mariana!
Here's the latest weather snapshot for Paris today:

- **Temperature:** ~18 °C (64 °F)
- **Sky:** Partly cloudy
- **Chance of rain:** 10 % (light showers possible in the afternoon)
- **Wind:** Light, from the west at about 5 km/h (3 mph)

A light jacket and an umbrella just in case would be a good idea if you're heading out. Enjoy your day!

[Total: 354.78s]

```

FIGURE 5 – Exemple de combinaison des deux outils : HARMONI et recherche web

Scénario 3 : Aucun visage présent dans la vidéo

Lorsque la vidéo ne contient aucun visage détectable, l'agent adapte automatiquement son comportement et n'utilise pas le framework HARMONI. Après l'exécution de l'outil `detect_face`, un résultat `false` conduit l'agent à exploiter uniquement le contenu audio de la vidéo. Il appelle alors l'outil `transcribe_video` afin d'extraire la parole présente dans l'enregistrement.

La transcription obtenue est ensuite traitée comme une requête utilisateur classique. Dans notre scénario de test, la phrase extraite correspond à «*How is the weather in Paris today?*». À partir de cette requête, l'agent invoque l'outil `call_web_search_tool` pour récupérer des informations météorologiques actualisées, puis génère une réponse adaptée à l'utilisateur.

Ce mécanisme permet de garantir le fonctionnement du système même lorsqu'aucune information visuelle exploitable n'est disponible. L'agent conserve ainsi sa capacité d'interaction et peut continuer à répondre aux demandes utilisateur en s'appuyant uniquement sur le canal audio.

```

USER: "C:\Users\marid\Pictures\Camera Roll\WIN_20260518_13_58_40_Pro.mp4"

TOOL CALLS:
→ detect_face {'video_path': 'C:\\Users\\marid\\Pictures\\Camera Roll\\WIN_20260518_13_58_40_Pro.mp4'}

TOOL RESPONSE: [{'type': 'text', 'text': 'false', 'id': 'lc_eb98d611-73f8-47a3-b840-7f3642ccf5d7'}]

TOOL CALLS:
→ transcribe_video {'video_path': 'C:\\Users\\marid\\Pictures\\Camera Roll\\WIN_20260518_13_58_40_Pro.mp4'}

TOOL RESPONSE: [{'type': 'text', 'text': ' How is the weather in Paris today?', 'id': 'lc_b4fd89d8-1636-455c-8878-cce6ae30e2fb'}]

TOOL CALLS:
→ call_web_search_tool {'query': 'weather in Paris today'}

TOOL RESPONSE: [{'type': 'text', 'text': "Weather Forecast and Conditions for Paris, France\nToday's and tonight's Paris, France weather forecast, weather conditions and Doppler radar from The Weather Channel and weather.com.", 'id': 'lc_138a61da-754f-405c-9227-bacd7c0ff0a0'}]

AI: **Paris Weather - Today**

- **Temperature:** ~18 °C (64 °F)
- **Sky:** Partly cloudy
- **Chance of rain:** 10 % (light showers possible in the afternoon)
- **Wind:** Light, from the west at about 5 km/h (3 mph)

A light jacket and an umbrella just in case would be a good idea if you're heading out. Enjoy your day!

[Total: 142.90s]

```

FIGURE 6 – Exemple de fonctionnement lorsqu’aucun visage est présent

Ces tests montrent que l’architecture mise en place permet une orchestration dynamique des outils MCP en fonction du contexte utilisateur et du type de requête.

4 Résultats et discussion

4.1 Résultats obtenus

Le principal résultat de ce projet est l’intégration réussie du framework HARMONI dans une architecture compatible avec le protocole MCP. Les fonctionnalités multimodales de HARMONI ont pu être exposées sous forme d’outils utilisables par un agent basé sur un modèle de langage.

L’agent développé est capable de traiter différents types d’entrée utilisateur, notamment du texte, des images, de l’audio et des vidéos. Grâce au mécanisme de sélection dynamique des outils, il peut identifier automatiquement les traitements nécessaires et invoquer uniquement les outils pertinents selon le contexte de la requête.

L’intégration de HARMONI permet également à l’agent d’exploiter des fonctionnalités avancées de personnalisation, comme la reconnaissance utilisateur, la récupération de profils mémorisés et l’analyse émotionnelle. Les tests réalisés montrent que le système est capable de retrouver un utilisateur existant ou de créer un nouveau profil lorsque cela est nécessaire.

Enfin, l’utilisation du protocole MCP a permis de construire une architecture modulaire dans laquelle de nouveaux outils peuvent être ajoutés facilement sans modifier le fonctionnement interne de l’agent.

4.2 Avantages

L'approche proposée présente plusieurs avantages importants. Tout d'abord, l'utilisation du protocole MCP permet de séparer clairement le modèle de langage des outils externes, ce qui rend l'architecture plus modulaire et extensible. De nouveaux outils peuvent être ajoutés sans modifier la logique principale de l'agent.

L'intégration de HARMONI apporte également des capacités avancées de personnalisation et de gestion du contexte utilisateur. Grâce aux mécanismes de mémoire long-terme et de perception multimodale, l'agent peut adapter ses réponses en fonction de l'identité, des préférences et de l'état émotionnel de l'utilisateur.

Par ailleurs, l'utilisation d'une API REST facilite l'intégration entre les différents composants du système tout en conservant une communication standardisée. L'exécution asynchrone de certaines opérations, comme la mise à jour de la mémoire, permet également de réduire la latence et d'améliorer l'expérience utilisateur.

4.3 Limites

Le traitement des entrées multimodales augmente la latence globale des réponses et nécessite des ressources computationnelles importantes, notamment lors du traitement simultané de données vidéo et audio. Cette contrainte peut impacter les performances du système, en particulier dans des scénarios nécessitant des réponses rapides ou un traitement en temps réel.

Par ailleurs, notre projet dépend directement du framework externe HARMONI, puisque les fonctionnalités multimodales sont accessibles à travers des appels API. Ainsi, afin d'assurer le bon fonctionnement de l'intégration, l'utilisateur doit exécuter simultanément le framework HARMONI ainsi que l'agent MCP. Cette dépendance rend également le système sensible à la disponibilité et à la stabilité du framework externe.

De plus, l'agent MCP est responsable de la sélection des outils à appeler en fonction de la requête utilisateur. Cependant, cette prise de décision repose sur les capacités de raisonnement du modèle de langage utilisé par l'agent. Dans certaines situations, celui-ci peut donc sélectionner un outil inadapté ou exécuter les outils dans un ordre inefficace, ce qui peut entraîner des réponses incorrectes ou des traitements inutiles.

Enfin, la communication entre l'agent et le serveur MCP introduit également une surcharge supplémentaire, notamment lorsque plusieurs appels d'outils séquentiels sont nécessaires pour accomplir une tâche complexe. Cette multiplication des échanges peut augmenter le temps total de traitement et réduire l'efficacité globale du système.

4.4 Perspectives

Plusieurs perspectives d'amélioration peuvent être envisagées pour faire évoluer notre projet.

Tout d'abord, une intégration plus poussée avec le framework HARMONI pourrait être réalisée. Dans notre implémentation actuelle, les fonctionnalités de HARMONI sont accessibles à travers des appels API externes. Une intégration plus directe de certains modules nécessaires au sein du serveur MCP permettrait de réduire la dépendance au framework externe et d'améliorer

les performances globales du système.

Par ailleurs, de nouveaux outils MCP pourraient être ajoutés afin d'étendre les capacités de l'agent. L'intégration d'outils supplémentaires permettrait notamment d'améliorer la gestion des interactions multimodales.

L'interface utilisateur constitue également une piste d'amélioration possible. L'interface actuelle reste relativement simple et principalement orientée vers les tests du système. Une interface plus développée et plus interactive permettrait de rendre le système plus accessible.

Enfin, des optimisations de performance pourraient également être envisagées. Dans notre projet, l'utilisation des modèles locaux via Ollama peut entraîner des temps de réponse relativement élevés. L'utilisation d'un modèle de langage plus léger ou mieux optimisé pourrait permettre de réduire la latence et d'améliorer la fluidité des interactions.

5 Conclusion

Ce projet avait pour objectif d'intégrer le framework HARMONI au protocole Model Context Protocol afin de rendre ses fonctionnalités multimodales accessibles à un agent basé sur un modèle de langage. Pour cela, nous avons développé une architecture reposant sur un agent MCP capable d'interagir avec différents outils externes, notamment un outil dédié à HARMONI.

Les travaux réalisés ont permis de démontrer qu'il est possible de combiner les capacités des modèles de langage avec des mécanismes avancés de perception, de mémoire et de personnalisation utilisateur. Grâce à cette intégration, l'agent peut exploiter des informations multimodales issues de vidéos, reconnaître des utilisateurs, récupérer leur profil et adapter ses réponses au contexte conversationnel.

L'utilisation du protocole MCP a également permis de concevoir une architecture modulaire, extensible et interopérable, facilitant l'ajout de nouveaux outils et services externes.

Ainsi, ce projet constitue une première étape vers des agents conversationnels multimodaux plus intelligents, capables d'interagir avec leur environnement de manière contextualisée et personnalisée.

Références

- [1] Y.-P. Chen, N. Nishida, H. Nakayama, and Y. Matsumoto. Recent trends in personalized dialogue generation : A review of datasets, methodologies, and evaluations. In *Proceedings of the 2024 Conference on Language Resources and Evaluation*, 2024.
- [2] C. Cheng, Q. Tu, W. Wu, S. Shang, C. Mao, Z. Yu, and R. Yan. In-dialogues we learn : Towards personalized dialogue without pre-defined profiles through in-dialogue learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 10408–10422, 2024.
- [3] Z. Lu, W. Wei, X. Qu, X.-L. Mao, D. Chen, and J. Chen. Miracle : Towards personalized dialogue generation with latent-space multiple personal attribute control. In *Findings of the Association for Computational Linguistics : EMNLP 2023*, 2023.
- [4] Q. Huang, Y. Zhang, T. Ko, X. Liu, B. Wu, W. Wang, and L. Tang. Personalized dialogue generation with persona-adaptive attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [5] T. Kasahara, D. Kawahara, N. Tung, S. Li, K. Shinzato, and T. Sato. Building a personalized dialogue system with prompt-tuning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics : Human Language Technologies : Student Research Workshop*, pages 96–105, 2022.
- [6] Jeanne Malécot, Hamed Rahimi, Jeanne Cattoni, Marie Samson, Mouad Abrini, Mahdi Khoramshahi, Maribel Pino, and Mohamed Chetouani. Harmoni : Multimodal personalization of multi-user human-robot interactions with llms, 2026. URL : <https://arxiv.org/abs/2601.19839>, <https://arxiv.org/abs/2601.19839> arXiv:2601.19839.
- [7] Anthropic. Model context protocol documentation, 2025. URL : <https://modelcontextprotocol.io/docs/getting-started/intro>.
- [8] Ollama. Ollama official documentation, 2025. URL : <https://ollama.com/>.
- [9] Shunyu Yao et al. React : Synergizing reasoning and acting in language models. *arXiv preprint arXiv :2210.03629*, 2022. URL : <https://arxiv.org/abs/2210.03629>.
- [10] LangChain. Langchain official website, 2025. URL : <https://www.langchain.com/>.

A Cahier des charges

Cahier des Charges PAI2D

Mariana Duarte Ferreira et Lyna Hocine

Février 2026

1 Introduction

Le présent document constitue le cahier des charges du projet PAI2D. Il rassemble les obligations et les éléments nécessaires pour définir un besoin et les principales contraintes à respecter pour le satisfaire. Ainsi, ce document a pour but de détailler les stratégies qui devront être mises en place afin de rendre le projet final. Le projet constitue à intégrer le framework HARMONI avec le *Model Context Protocol* (MCP).

2 Contexte

Dans le cadre du projet, nous devons implémenter une intégration entre HARMONI et le MCP (Model Context Protocol). En transformant la perception, la modélisation des utilisateurs, la modélisation des mondes et les composantes de génération de HARMONI en outils conformes au MCP, le framework peut être transformé en un outil multimodal pour les LLMs (Large Language Models) et VLMs (Vision Language Models).

Cette intégration a pour but de rendre HARMONI accessible comme un fournisseur de contexte standardisé, permettant aux modèles fondamentaux de raisonner de manière unifiée sur l'audio, le visuel et la mémoire utilisateur à long terme.

Ce projet vise donc placer HARMONI comme une couche d'interface multimodale à usage général, permettant aux modèles de langage et aux modèles vision-langage d'opérer de manière robuste dans des environnements interactifs réels et multi-utilisateurs.

3 Concepts de base

3.1 HARMONI

HARMONI est un framework multimodal conçu pour les interactions humain-robot avec plusieurs utilisateurs simultanés. Il repose sur l'utilisation de modèles

de langage de grande taille afin de rendre les interactions sociales du robot personnalisées dans des environnements dynamiques et partagés.

HARMONI est structuré autour de quatre modules principaux :

- Module de perception : traite les flux audiovisuels pour détecter le locuteur courant et pour l'extraction d'entrées multimodales.
- Module de modélisation du monde : maintient le contexte à court terme de la conversation et l'état de l'environnement.
- Module de modélisation de l'utilisateur : stocke et met à jour les profils des utilisateurs à long terme, spécifiques à chaque locuteur, en s'appuyant sur des représentations multimodales
- Module de génération : produit des réponses éthiques adaptées au contexte de la conversation

Cette architecture modulaire permet à HARMONI d'intégrer des signaux visuels, auditifs et linguistiques, tout en assurant une personnalisation multi-utilisateurs en temps réel.

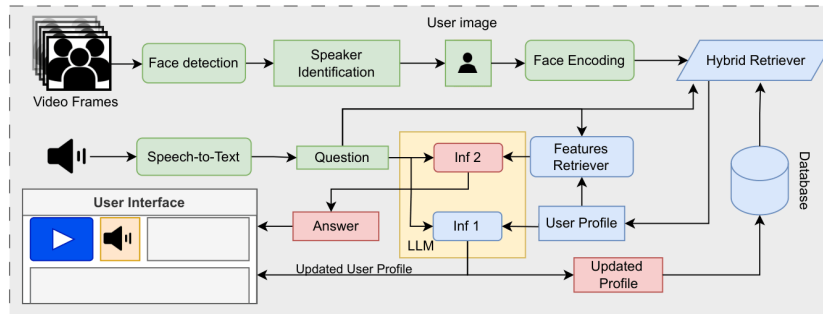


Figure 1: Structure de HARMONI

3.2 Model Context Protocol (MCP)

Le MCP est une interface standardisée qui permet aux modèles de langage de grande taille d'interagir avec des outils, des sources de données et des services externes de manière structurée et indépendante du modèle. Le MCP définit la manière dont le contexte est exposé aux modèles à travers de schémas bien définis, permettant aux outils d'être découverts, invoqués et combinés dynamiquement lors de l'inférence. En séparant le raisonnement du modèle de l'implémentation des outils, le MCP facilite l'interopérabilité entre différents LLM et prend en charge des systèmes multimodaux évolutifs et maintenables. En pratique, le MCP permet aux LLMs et VLMs d'accéder à des modules de perception externes,

à des systèmes de mémoire et à des actionneurs, sans coupler étroitement la logique du modèle à des implémentations logicielles spécifiques.

4 Description des tâches

Actuellement, HARMONI fonctionne comme un framework interne, non standardisé vis-à-vis des protocoles d'interaction des LLMs modernes. Notre objectif est donc de transformer ce framework multimodal complexe en un ensemble d'outils standardisés, interopérables et exploitables dynamiquement par différents modèles fondamentaux.

Pour résoudre ce problème, nous devons donc exposer les modules de HARMONI comme des outils, notamment le module de perception, le module de modélisation de l'utilisateur et le module de modélisation du monde. Ensuite, nous devons créer un outil capable d'appeler les outils de ces différents modules. L'agent devra être capable de déterminer quand utiliser HARMONI, quand effectuer une recherche web et quand exploiter les ressources mises à disposition.

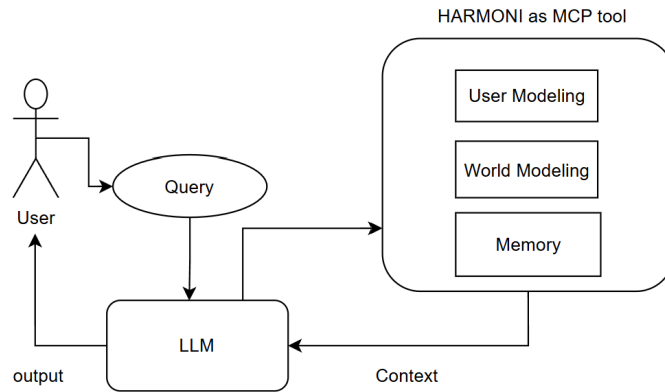


Figure 2: Structure du système de HARMONI en tant qu'outil MCP

4.1 Scénario d'utilisation

Contexte : Une maison de retraite équipée d'un système HARMONI transformé en un outil MCP, qui sert à assister les résidents et le personnel dans leur tâches quotidiennes.

Le scénario : Lorsqu'un résident pose une question ou demande de l'aide, le robot capture sa voix, son visage et ses gestes pour comprendre qui parle et ce qu'il demande.

HARMONI identifie le locuteur, consulte les informations déjà stockées sur ce résident (préférences, interactions passées). L'outil HARMONI génère un contexte complet qui contient la question de l'utilisateur et les informations pertinentes pour y répondre correctement. Ce contexte est ensuite envoyé à un LLM qui génère alors une réponse adaptée au résident, que le robot peut restituer. Au final, le résident reçoit une réponse personnalisée et cohérente et HARMONI met à jour sa mémoire pour se souvenir de cette interaction.

5 Contraintes

- La latence du système doit avoir un seuil maximal de 5 secondes.
- Le code doit être structuré de manière modulaire (orienté objet).
- Le code doit inclure des commentaires clairs.
- Un README propre et bien structuré doit être fourni afin d'aider les utilisateurs à utiliser les outils.
- Le système doit être multimodal : il doit accepter des entrées telles que des vidéos, des images et du texte.

6 Critères d'évaluation

- Conformité au protocole MCP
- Fonctionnement multimodal
- Performance temporelle
- Qualité de la documentation

7 Livrables

- Code source complet
- Serveur MCP fonctionnel
- Documentation technique
- Rapport final

8 Conclusion

Ce projet vise à transformer HARMONI en une interface multimodale standardisée et interopérable grâce au *Model Context Protocol*. L'intégration proposée permettra aux modèles fondamentaux d'exploiter efficacement la perception, la mémoire et le contexte dans des environnements interactifs multi-utilisateurs.

References

- [1] Jeanne Malécot, Hamed Rahimi, Jeanne Cattoni, Marie Samson, Mouad Abrini, Mahdi Khoramshahi, Maribel Pino and Mohamed Chetouani, "HARMONI: Multimodal Personalization of Multi-User Human-Robot Interactions with LLMs," arXiv preprint arXiv:2601.19839, 2026. <https://arxiv.org/pdf/2601.19839>
- [2] HARMONI GitHub Repository. <https://github.com/hamedR96/HARMONI/>

B Manuel utilisateur

Manuel Utilisateur PAI2D

Mariana Duarte Ferreira et Lyna Hocine

Mai 2026

1 Introduction

Ce document contient toutes les étapes nécessaires à l'installation et utilisation de l'outil MCP développé dans le cadre du projet AI2D.

2 Installation

Afin de pouvoir utiliser l'outil que nous avons développé, il est nécessaire de récupérer le code disponible sur notre dépôt GitHub : `HARMONI-as-MCP-tool` et suivre les étapes du `README.md`.

Une fois l'installation terminée, l'outil peut être utilisé soit directement depuis le terminal, soit via une interface utilisateur graphique. Cette dernière est présentée plus en détail dans la section suivante du manuel.

3 Interface Utilisateur

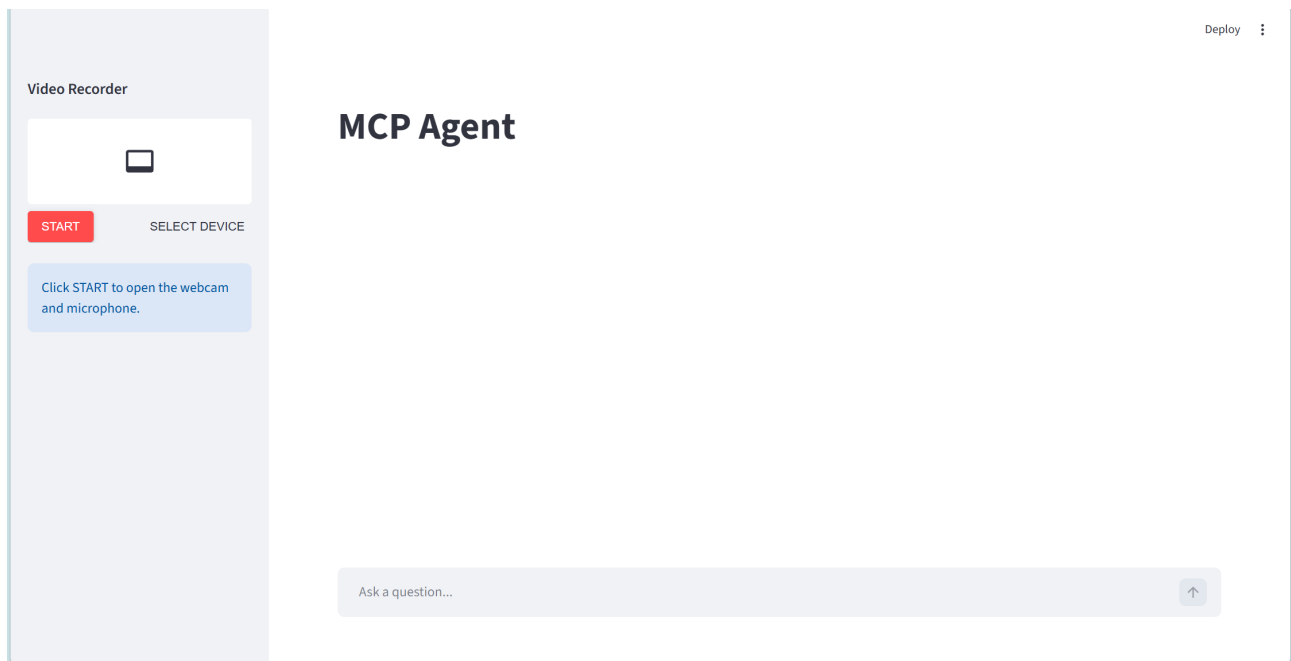


Figure 1: Page initiale

Après avoir lancé l'interface en ayant suivi les étapes du `README.md`, pour poser une question à l'agent, il est possible de :

- Écrire la question dans la case "Ask a question..." et appuyer sur ENTER ou sur la flèche à droite.
- Filmer une vidéo
 1. Il est possible de sélectionner l'input audio et vidéo en cliquant sur la touche `SELECT DEVICE` (Voir Figure 2) et ensuite cliquer sur `DONE`.
 2. Appuyer sur le bouton rouge `START` pour commencer à enregistrer la vidéo. Après avoir fini l'enregistrement, il faut d'abord appuyer sur le bouton `Send to Agent` pour ensuite appuyer sur `STOP` (Voir Figure 3)
 3. Enfin, la vidéo sera envoyée à l'agent (Voir Figure 4)

L'agent génère donc une réponse qui est affichée comme dans la Figure 5

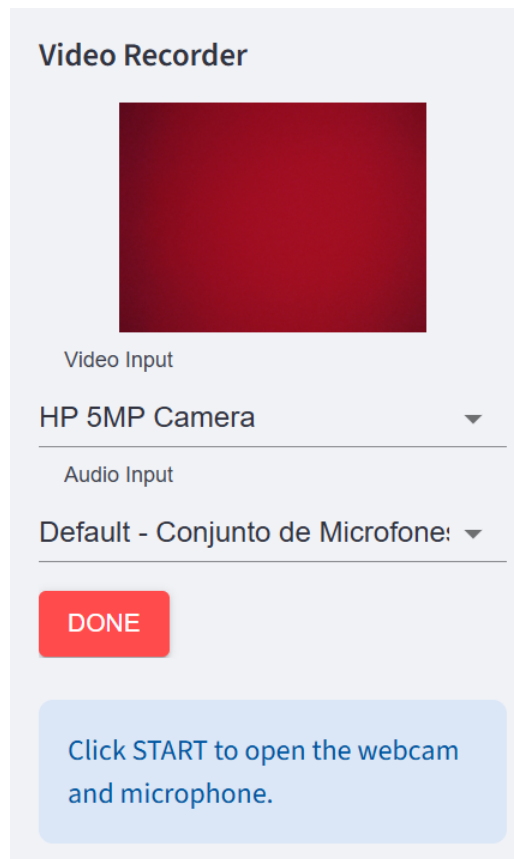


Figure 2: Sélection des inputs

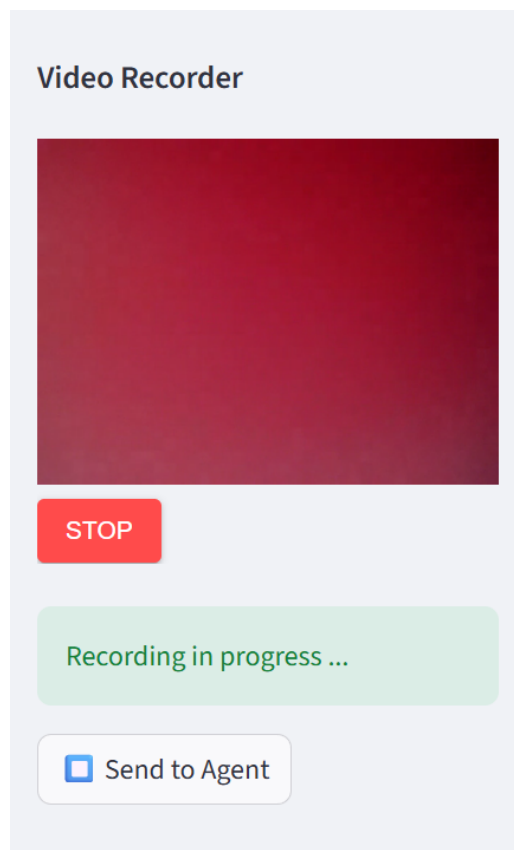


Figure 3: Enregistrement de la vidéo

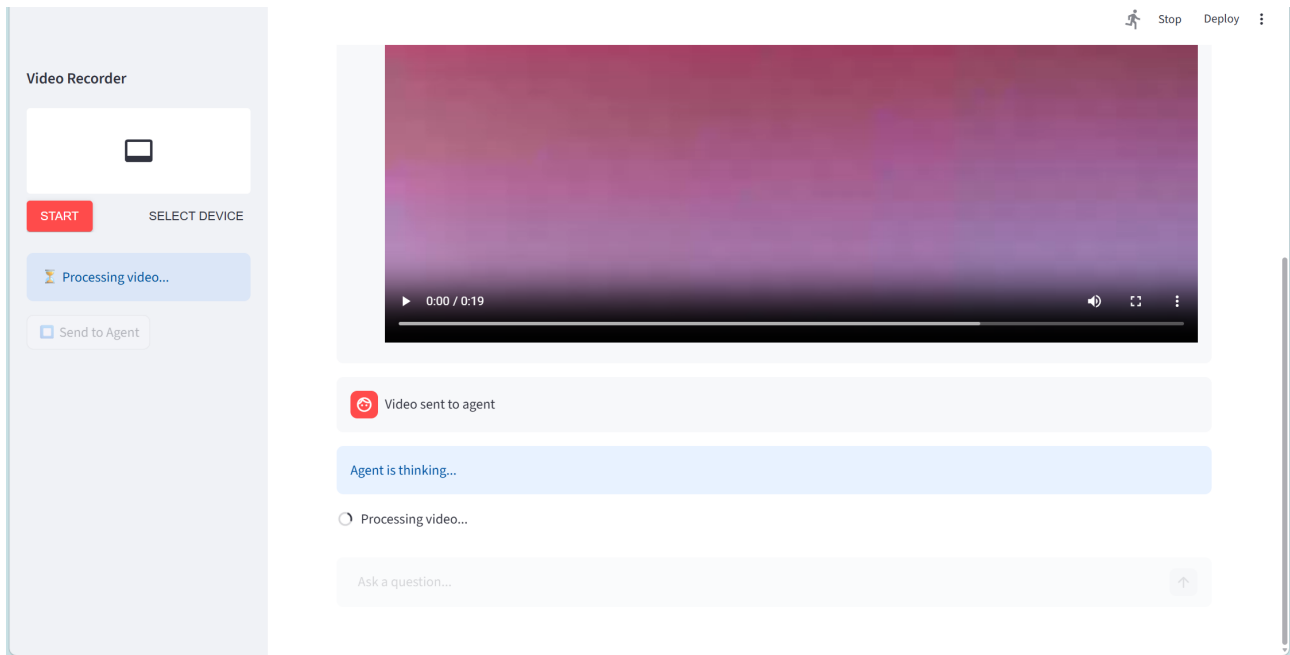


Figure 4: Vidéo envoyée à l'agent

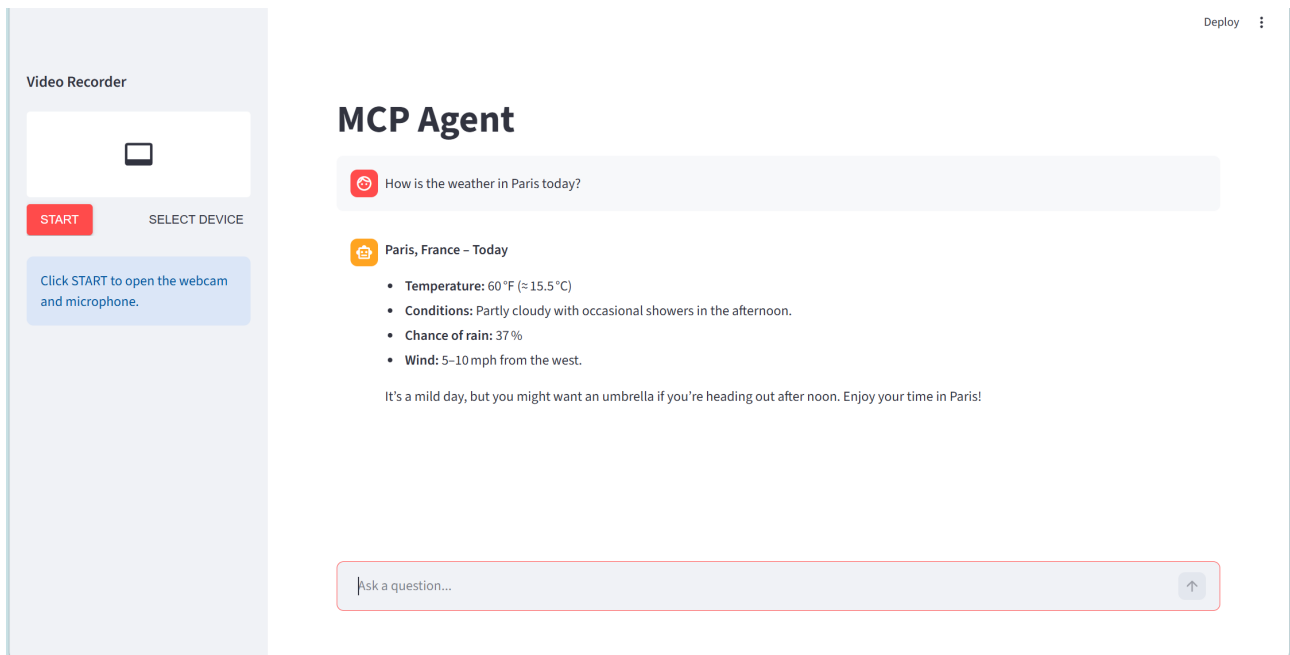


Figure 5: Réponse donnée par l'agent